



TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
NATIONAL COLLEGE OF ENGINEERING

A
MID-TERM PROJECT REPORT
ON
BASOBAS: A MOBILE RENTAL ACCOMMODATION FINDER

SUBMITTED BY:

NITESH PANT (NCE080BCT024)

RAJESH BHANDARI (NCE080BCT027)

SARWAGYA ACHARYA (NCE080BCT038)

UTSAV BHATTARAI (NCE080BCT048)

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

LALITPUR, NEPAL

JULY, 2026

CONTENTS

List of Figures	iii
List of Tables	iv
List of Abbreviations	v
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	3
1.3 Aims and Objectives	4
1.3.1 Aim	4
1.3.2 Objectives	4
1.4 Scope	4
2 Literature Review	7
2.1 Related Work	7
2.2 Related Theory	11
2.2.1 Two-Sided Platforms and Matching	11
2.2.2 Relational Data Modelling	11
2.2.3 Geospatial Distance (Haversine Formula)	12
2.2.4 Rating Aggregation	12
2.2.5 Basic Security and Privacy	13
3 Methodology	14
3.1 System Architecture	14
3.1.1 Architecture Design	14
3.1.2 Use Case Diagram	15
3.2 Implementation Details	16
3.2.1 Frontend / Mobile App Development	16
3.2.2 Backend / Database	17
3.3 Module Implementation	17

3.3.1	Authentication and Onboarding Module	17
3.3.2	Property Listing Management Module	18
3.3.3	Property Discovery Module	19
3.3.4	Visit Request Workflow Module	19
3.3.5	Landlord Experience Module	20
3.3.6	Tenant Experience Module	20
3.3.7	Pro Plan and Payment Module	20
3.3.8	Notification System Module	22
3.3.9	Ratings and Reviews Module	22
3.3.10	KYC Verification Module	22
3.3.11	AI Recommendation Module	23
3.3.12	Profile and Account Management Module	23
3.4	Testing Strategy	25
4	Tasks Accomplished	26
4.1	Product Planning and Documentation:	26
4.2	UI/UX Design:	26
4.3	User Authentication:	28
4.4	Onboarding Flow Implementation:	28
4.5	Dual-Role Account System:	29
4.6	Database Architecture with Supabase:	30
4.7	eSewa Payment Gateway Integration:	30
5	Tasks Remaining	32
	References	33

LIST OF FIGURES

3.1	System Architecture of BasoBas	15
3.2	Use Case Diagram for BasoBas	16
3.3	Authentication and Onboarding Workflow	18
3.4	Visit Request Status Lifecycle	20
3.5	eSewa Payment Transaction Flow	21
3.6	Confidence-Based Review Routing	23

LIST OF TABLES

2.1 Core Relational Entities for BasoBas 11

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
GPS	Global Positioning System
HTTPS	Hypertext Transfer Protocol Secure
JWT	JSON Web Token
KYC	Know Your Customer
LBS	Location-Based Services
MERN	MongoDB, Express.js, React, Node.js
MVP	Minimum Viable Product
NPR	Nepalese Rupee
OCR	Optical Character Recognition
OTP	One-Time Password
PHP	Hypertext Preprocessor
RLS	Row-Level Security
SDK	Software Development Kit
SMS	Short Message Service
SUS	System Usability Scale
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UX	User Experience
XP	Extreme Programming

1. INTRODUCTION

1.1 Background

Rental accommodation is a housing arrangement in which a property owner allows another person to live in a room or property in exchange for rent. It is one of the most common ways people find a place to live, especially students, working professionals, and families who move to a new area for education, work, or other reasons. Renting provides a practical option for people who need a place to stay without buying a house. The rental process mainly involves two people: the landlord and the tenant. The landlord offers a room or property for rent by deciding details such as the rent, available facilities, availability, and tenant preferences. The tenant looks for a room that matches their budget, preferred location, and other needs. When both sides have complete and accurate information, it becomes easier for tenants to find a suitable place and for landlords to find suitable tenants.

In Nepal, finding a rental room is still largely an informal process. Every year, many students and young professionals move to the Kathmandu Valley from different parts of the country for higher education and employment, making the search for accommodation one of their first priorities [1]. Most people depend on recommendations from friends and relatives, social media groups, local advertisements, or real estate agents to find available rooms. Because information is scattered across different sources, tenants often spend a lot of time contacting landlords and visiting multiple properties before finding a suitable room. It is also common to discover that a room is no longer available or that the rent, facilities, or tenant preferences are different from what was initially advertised. Nepal has online rental platforms such as KothaBhada and GharBheti that help landlords advertise available properties. These platforms have made rental listings more accessible, but they are designed to cover many types of properties rather than focusing on rental rooms. As a result, details that many tenants consider important, such as whether a room is suitable for students, bachelors, families, or working professionals, are not always available as searchable filters. In many cases, this information is only mentioned in the property description, making it harder for users to compare listings and find rooms

that meet their requirements. Similar issues have been identified in studies of online rental platforms, where incomplete or unstructured listing information makes the search process less effective [2].

Location is another important factor when choosing a rental room. Students often prefer rooms close to their college or university, while working professionals usually look for accommodation near their workplace. However, searching for nearby rooms is not always straightforward. Users often have to browse listings one by one instead of searching within a selected distance from a specific location.

Communication between landlords and tenants is also mostly handled through phone calls or messaging applications. After finding a suitable room, tenants need to contact the landlord to ask whether it is still available and arrange a visit. Landlords, on the other hand, have to manage every inquiry separately, especially when several people are interested in the same property. There is no simple way for either side to keep track of visit requests or their status.

To help improve this process, BasoBas is proposed as a mobile application for rental room discovery in the Kathmandu Valley. The application allows landlords to create property listings with clear information about the room, while tenants can search using filters such as location, budget, room type, facilities, and tenant preferences. It also includes features for nearby search, visit requests, KYC verification, and ratings and reviews.

The application is designed to keep the rental process simple. A tenant can browse available rooms, apply filters, view property details, and request a visit. The landlord can review the request and choose to accept, reschedule, or decline it. After visiting the property, tenants can leave ratings and reviews, helping future users make better-informed decisions.

1.2 Problem Statement

Tenants relocating to cities such as Kathmandu for study or work face significant difficulties in finding suitable rental accommodation. The discovery process remains informal and broker-dependent, while listing information is often incomplete or unstructured. Landlords may prefer specific tenant categories, but these rules are not consistently transparent on existing portals. This leads to mismatch, repeated travel, wasted time, and avoidable costs.

The major problem points are:

- (i) Heavy dependence on brokers and informal contacts, with high commission and limited accountability.
- (ii) Lack of clear, filterable tenant-acceptance criteria such as “students allowed” and “bachelors allowed.”
- (iii) Weak support for proximity-based search around current location or selected reference points.
- (iv) Limited workflow support for booking/visit requests and transparent status updates.
- (v) Absence of a strong trust signal for new tenants who need reliable reputation information.

Therefore, a focused location-based mobile platform is needed to support structured matching between tenants and landlords.

1.3 Aims and Objectives

1.3.1 Aim

The main aim of this project is to develop a location-based mobile application that makes it easier for people to find rental rooms in the Kathmandu Valley. The application provides a single platform where tenants can search for rooms and landlords can list their properties, making the rental process simpler and more organized for both.

1.3.2 Objectives

1. To allow landlords to add and manage room listings by providing details such as rent, facilities, address, location, tenant preferences, and availability.
2. To help tenants find rental rooms in the valley using filters such as location, budget, room type, room features, facilities, tenant preferences (students, families, working professionals, or anyone), availability, and ratings.
3. To provide a nearby search feature that allows users to find rooms within a selected distance from their current location or a location they choose on the map.
4. To develop a visit request system where tenants can request to visit a property and landlords can accept, reschedule, or decline the request.
5. To implement a KYC verification process for landlords and tenants to verify their identities.
6. To include a rating and review feature that allows tenants to share their experience and help others when choosing a rental room.

1.4 Scope

BasoBas is a mobile application developed to simplify the process of finding and listing rental rooms and small flats. The project is mainly intended for students, working professionals, and small families looking for affordable rental accommodation. The scope of the project is described below.

- **Application Scope:** The application allows users to browse, search, and list rental properties such as rooms and small flats.

- **User Scope:** The system is designed for two main types of users: tenants and landlords. A basic admin panel is included for managing the system.
- **Platform Scope:** The application is developed for Android devices using React Native, making it possible to support other mobile platforms in the future.
- **Functional Scope:** The system provides features such as user registration, property listing, property search with filters, nearby property search, visit request management, KYC verification, and property reviews and ratings.
- **Data Scope:** User information, property details, visit requests, and reviews are stored in a PostgreSQL database managed through Supabase.
- **Project Limitations:** The current version of the application does not support online rent payments, digital rental agreements, or dispute resolution between landlords and tenants.

The BasoBas prototype operates within a framework of basic legal and ethical responsibility, and the design of the system actively incorporates these principles rather than treating them as afterthoughts.

With respect to location data, the system requests GPS permission from the user only when the nearby-search feature is activated. The application does not collect location data in the background, does not track user movement across sessions, and provides a manual area-search fallback for users who decline location permission. This aligns with the principle of purpose limitation, which requires that personal data be used only for the specific, declared purpose for which it was collected [3].

For user identity and verification, BasoBas includes a KYC (Know Your Customer) step in the onboarding flow. Landlords are required to submit identity documentation before their listings go live. Tenants have the option to complete KYC verification, which builds additional trust in the platform. Submitted documents are stored securely through Supabase Storage and are accessible only to the platform administrators for verification purposes, not to other users. This directly addresses the fraud and trust concerns that prior work on peer-to-peer rental platforms has identified as significant barriers to adoption [4].

User authentication is handled through Supabase Auth, which provides secure session management and hashed credential storage. Row-Level Security (RLS) policies on the Supabase PostgreSQL database ensure that users can only access and modify their own records. Tenants cannot modify landlord listings, and landlords cannot alter tenant booking records they do not own.

Reviews and ratings are restricted to tenants who have completed a visit, which prevents fake or malicious review submissions and preserves the integrity of the reputation system. [5].

All data collected through BasoBas is used solely for the purpose of facilitating property discovery and the tenant-landlord interaction workflow. No data is sold, shared with third parties, or used for advertising. A basic privacy notice is provided at registration, and users may request deletion of their accounts and associated data.

2. LITERATURE REVIEW

2.1 Related Work

To better understand what has already been done in the area of online room-finding and rental platforms, a number of related research papers were read and reviewed. Each paper was studied to understand what problem they were trying to solve and how they built their solution. The results they achieved and what was still missing in their work was also noted. This helped in identifying the gaps that the present work aims to fill. A summary of each reviewed paper is presented below.

Diaz et al. developed RoomMeet, a rental mobile application that targets a variety of users, such as students, families, and working professionals [4]. The platform was developed using an Extreme Programming (XP) methodology, where user stories were gathered through surveys guided iterative low-fidelity prototyping. Data for the study came from a 50-respondent survey, of which 74% had previously searched for rooms or apartments to rent, along with System Usability Scale (SUS)-style ratings of the prototype. Results showed that users highly accepted the idea, with most respondents rating the prototype as better than existing rental search alternatives and identifying search filters as the most useful feature. Yet, there was no actual deployment and testing on the large scale, since the evaluation was limited to a survey-based prototype rather than a functioning system with live landlords and tenants.

Farooq et al. proposed RoomMate, an AI-optimized space rental system that is mainly intended for student housing and room-sharing purposes [6]. The system was built using an Agile Software Development Life Cycle and supplemented by a Dialogflow-based AI chatbot for natural-language room search and FAQ handling. No empirical dataset is mentioned. Instead, they used requirement gathering through interviews and use-case/user-story development to define system functionality. The final product was a role-based platform offering verified listings, roommate matching, and chatbot-assisted search, which was validated through unit integration end-to-end, and performance testing. Still, the paper does not report any live-user evaluation, which means that the actual accuracy of the AI chatbot and the platform's real-world trust-building claims remain

unverified beyond internal testing.

Hess et al. examined neighborhood representation on internet rental housing platforms [7]. In their study, they compared Craigslist and Apartments.com about space, platform type, and metropolitan segregation. The researchers used web-scraping to collect the listings with the help of Helena and Python scripts. The dataset covered 847,068 Craigslist listings and 462,073 Apartments.com listings across the 100 most populated U.S. metropolitan areas from January to August 2019. The results revealed that high-income, highly educated, and low-poverty neighborhoods are consistently overrepresented on both platforms, while Black and Latinx neighborhoods are systematically underrepresented on Craigslist in more segregated metropolitan areas. One limitation of the study is that it only focuses on two platforms and also does not capture real-time changes in actual rental availability.

Zaman et al. presented an interactive web platform and Android application that facilitate a room-sharing scheme for students in Bangladesh facing accommodation shortages during admission-test season [8]. The system was built using PHP for the website and a corresponding Android application, with a layered moderation structure of global administrators and area-specific local moderators, and was evaluated through load testing and a user survey. Load-testing data showed that 50 virtual users produced 10,803 requests within five minutes of testing. Results indicated suggesting general user satisfaction with the prototype. The work, however, lacked working payment or fully automated verification mechanisms. They recognized the interface was not fully user-friendly and the platform was limited to the Android operating system only.

Indrayani and Istikomah created a web-based platform for boarding-house rental services in Pati Regency, Indonesia, focusing on information architecture and booking workflow [2]. The system followed the Waterfall development model, with requirements gathered through semi-structured interviews and field observations. Then, through the use of UML, the use case and class diagrams were modeled. At last, black-box testing was conducted for object validation. No large external dataset was used. Instead, the data consisted of qualitative interview and observation records used to derive functional and non-functional requirements. The results showed that all tested modules passed black-box testing without critical defects, confirming the functional reliability of the system. A clear limitation is that the system depends on manual payment confirmation rather

than an automated payment gateway.

Tyagi et al. proposed RoomRover, a smart room rental platform that uses conversational AI for helping users book rooms [9]. The platform was built on the MERN stack, integrating an AI-powered chatbot. This chatbot was created through different stages like gathering the requirements, designing the system, implementing it, and testing it. The study does not cite an external dataset but it shows the performance metrics of the system that was obtained after deploying the chatbot. The findings showed that the chatbot was responding under 3 seconds, the system was available 99.9% of the time, 85% of users' requests were resolved without human assistance, and 30% due to personalized recommendations. However, that the system lacks wider adoption testing, and reliance on AI chatbots may not be universally accepted by users who prefer human assistance.

Balpande et al. developed a web-based room-searching platform called Room Dhundo, targeting tenants and room owners in India [10]. The core objective of their work was to eliminate broker dependency and provide a centralized, affordable, and transparent accommodation-finding system. To achieve this, they built the platform using Python for the backend, enhanced it with Google Geolocation APIs, a real-time chat module, and an AI-powered chatbot to handle frequently asked questions. The platform also integrated a mess service finder and a carpooling feature to address students' broader daily needs. Results demonstrated that the system successfully connected tenants and landlords in a more efficient and transparent manner, reducing communication delays through the built-in chat feature. However, the work doesn't include any formal user evaluation or performance benchmarking.

Wakodikar et al. presented the design and implementation of a web-based To-Let (Room Finder) application intended to automate and streamline rental property management in urban India [11]. Their main reason was to eliminate the manual processes involving brokers that were not only slow but were also very prone to errors. The methodology followed a structured sequential approach supporting full CRUD (Create, Read, Update, Delete) operations. A comparative analysis presented in the paper showed that the proposed system outperformed manual methods and existing partial-digital systems in terms of data organization, automation, processing speed, and scalability. However, the application remains a basic prototype; it does not include advanced features such

as search filters by preference, geolocation-based discovery, user reviews, or a communication channel between tenants and landlords, which significantly limits its practical usability in a real-world urban rental scenario.

Fensen, Dewayani, and Wasino designed a website application specifically for locating and reserving houses and boarding rooms for rent, targeting students enrolled in universities in Jakarta, Indonesia [12]. They identified the increasing demand for housing near universities as the biggest reason behind the rising rental prices. The data were gathered through semi-structured interviews, focus group discussions with students, and surveying, whereas primary data consisted of photographs of real properties. A notable technical feature was the implementation of the Haversine formula to calculate the distance of a listed property from the user's current or selected location, enabling search results filtered by radius and administrative region. The application also supported an online booking module, allowing users to reserve rooms directly and track their rental history. Despite these strengths, the system was limited to a specific geographic area around one university campus, and the study did not report any quantitative evaluation of user satisfaction or system accuracy, leaving questions about scalability and broader adoption unanswered.

Okechi John Uchechukwu and Aisha Abdulrazaq Shehu highlighted an often-neglected aspect of housing, i.e. the problems experienced by foreign migrants, mainly Africans, when looking for rental housing in Russia [13]. Their work introduced Afro-Kiva (later rebranded as Miradom), a culturally aware, multilingual digital platform designed to reduce language barriers, counter discriminatory practices by landlords, and provide a trusted, verified rental experience. The methodology was comprehensive, combining literature review, online surveys of African students and expatriates, interviews with real estate agents, focus group discussions, and competitor analysis of existing Russian platforms such as Avito and Sutochno. Quantitative findings from their surveys revealed that over 60% of African students experienced serious difficulties in finding culturally inclusive housing. The platform was developed following user-centered design principles and proposed features including verified property listings, multilingual support, a dedicated customer support center, and value-added services such as noise monitoring and cleaning. However, it was just a preprint and prototype stage; the platform had not been deployed or tested at scale at the time of publication.

2.2 Related Theory

2.2.1 Two-Sided Platforms and Matching

BasoBas functions as a two-sided digital platform that connects two user groups: tenants and landlords [14]. The platform’s value depends on efficient matching between tenant preferences and landlord constraints. Tenants typically optimize for rent, location, room type, and acceptance policy, while landlords optimize for appropriate occupant profile and reliable tenancy. Structured listing attributes convert these preferences into filterable parameters and improve match quality.

2.2.2 Relational Data Modelling

The core BasoBas entities are well suited to relational representation. Primary keys uniquely identify records, while foreign keys enforce relationships and referential integrity. Normalized tables reduce redundancy and support reliable updates [15].

Table 2.1: Core Relational Entities for BasoBas

Entity	Primary Key	Important Attributes / Foreign Keys
Profiles	clerk_id	phone, full_name, active_role, city
LandlordProfile	clerk_id (FK)	verification_status, avg_rating, total_reviews, is_phone_shared_default
KYCSubmission	id	clerk_id (FK), document_type, status, submitted_at, reviewed_at
Listing	id	landlord_clerk_id (FK), title, description, property_type, status, price, address, city, locality, latitude, longitude, bedrooms, bathrooms, furnished_status, parking, amenities, available_from
ListingImage	id	listing_id (FK), image_path, display_order, is_cover

VisitRequest	id	listing_id (FK), tenant_clerk_id (FK), landlord_clerk_id (FK), requested_date, requested_time_slot, status
SavedListing	id	tenant_clerk_id (FK), listing_id (FK), saved_at
Product	id	name, price, duration_months, active
Transaction	id	clerk_id (FK), product_id (FK), total_amount, status, esewa_ref_id
UserPass	id	clerk_id (FK), product_id (FK), transaction_id (FK), starts_at, expires_at, status

2.2.3 Geospatial Distance (Haversine Formula)

For nearby-room functionality, BasoBas computes approximate surface distance between user/reference coordinates and property coordinates using the Haversine formula [16]. Let user coordinates be (ϕ_1, λ_1) and property coordinates be (ϕ_2, λ_2) in radians:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cos(\phi_2) \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (2.1)$$

$$c = 2 \arctan 2 \left(\sqrt{a}, \sqrt{1-a} \right), \quad d = R c \quad (2.2)$$

where $\Delta\phi = \phi_2 - \phi_1$, $\Delta\lambda = \lambda_2 - \lambda_1$, and $R \approx 6371$ km. A property is included in nearby results if $d \leq r$, where r is the selected radius.

2.2.4 Rating Aggregation

A basic reputation layer can be implemented through aggregated tenant ratings. If a property has ratings $r_1, r_2, \dots, r_n$, the average rating is computed as:

$$\bar{R} = \frac{1}{n} \sum_{i=1}^n r_i \quad (2.3)$$

This aggregate score helps users rank and filter listings and aligns with standard online marketplace reputation design principles [5]. Related feedback-system analysis also supports this approach [17].

2.2.5 Basic Security and Privacy

Security and privacy principles considered in BasoBas include:

- Authentication and authorization so users can only perform allowed actions.
- Secure password handling in hashed form if backend authentication is used.
- Explicit location permission before enabling nearby search mode.
- Limited location usage for search and navigation only, without continuous background tracking.
- Manual area-search fallback when GPS permission is denied.

These controls reflect known LBS privacy considerations discussed in prior work [3]. Additional legal discussion is provided in [18].

3. METHODOLOGY

3.1 System Architecture

3.1.1 Architecture Design

BasoBas adopts a multi-layer client–server architecture with separate presentation, application, administration, and data layers to ensure scalability, security, and maintainability.

- **Presentation Layer:** Consists of the BasoBas mobile application that provides user interfaces for authentication, property listing, property discovery, visit requests, notifications, AI recommendations, ratings, and profile management for both tenants and landlords
- **Administration Layer:** Consists of the web-based Admin Portal responsible for user management, landlord KYC verification, and review management. It integrates an AI-assisted KYC workflow using OCR, document validation, fraud detection, and a verification decision engine, while allowing administrators to manually review and approve flagged verification requests.
- **Application and Service Layer:** Implements the core business logic using Supabase services, including authentication, database operations, realtime synchronization, server-side Edge Functions, and secure file storage. External services such as Google Maps SDK and Clerk Authentication are integrated to provide location-based functionality and secure user authentication.
- **Data Layer:** Uses Supabase PostgreSQL to store structured application data such as users, properties, visit requests, reviews, subscriptions, and KYC records. Supabase Storage securely manages unstructured data, including KYC documents, user avatars, and property images and videos.

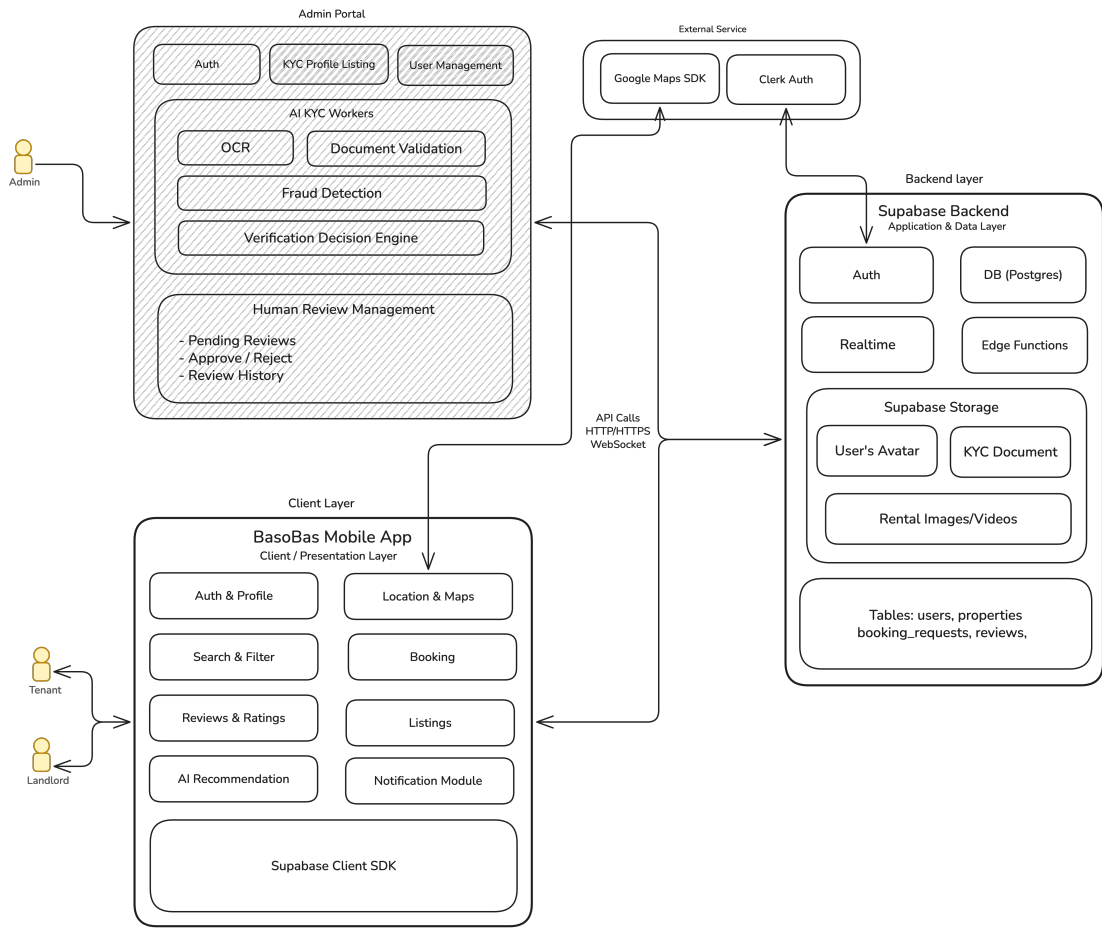


Figure 3.1: System Architecture of BasoBas

3.1.2 Use Case Diagram

The principal actors are Tenant and Landlord, with optional Admin for future extension. Tenant use cases include registration/login, search/filter, nearby discovery, booking request submission, and review submission. Landlord use cases include listing creation and management, request decision handling, and availability updates.

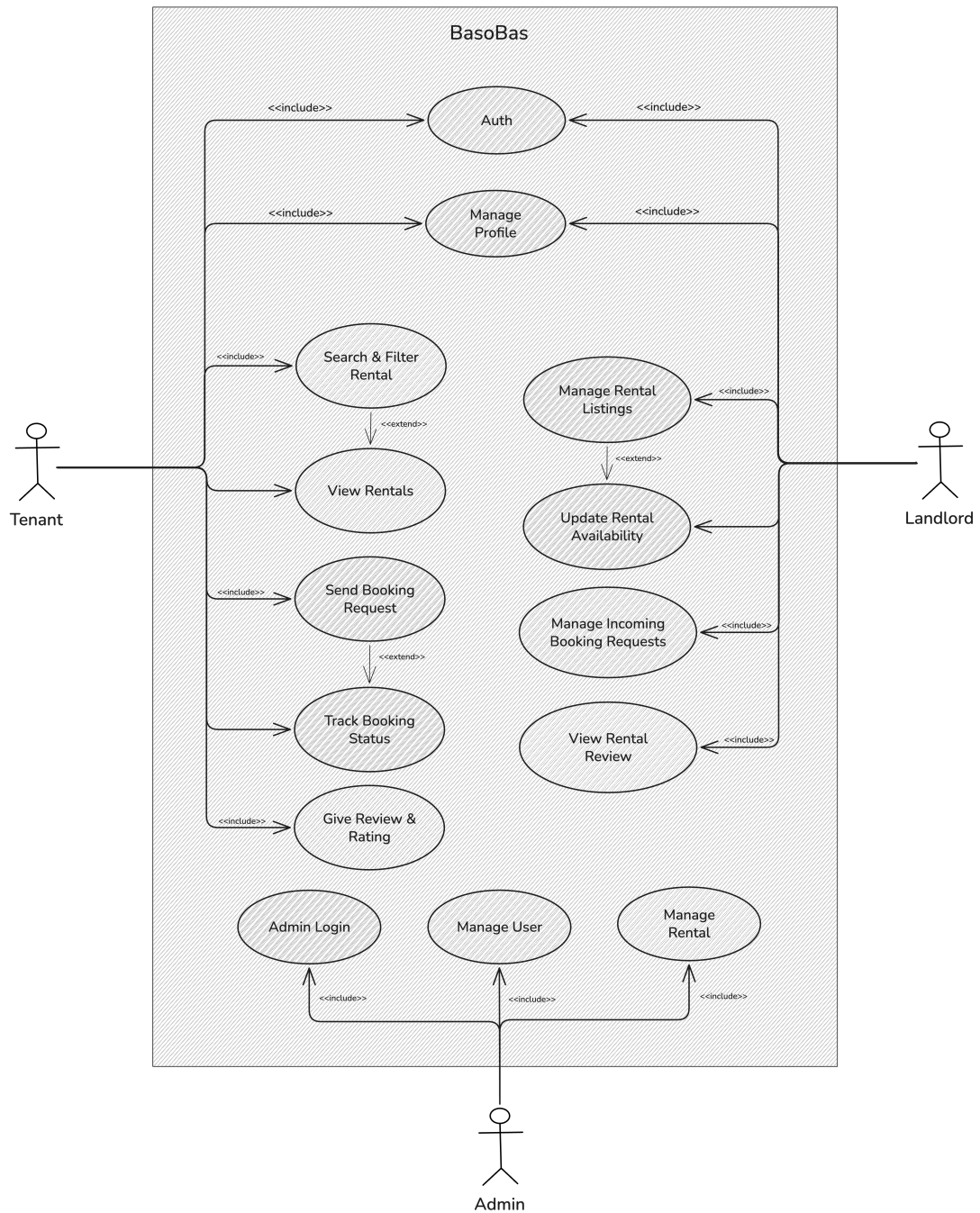


Figure 3.2: Use Case Diagram for BasoBas

3.2 Implementation Details

3.2.1 Frontend / Mobile App Development

The mobile frontend will be implemented using React Native with Expo and deployed through Expo's managed workflow. This provides a single codebase for Android-first delivery with future iOS support. The implementation covers role-based tenant and

landlord flows, including authentication, search and filtering, property details, booking requests, dashboard management, and review submission. Navigation will handle authenticated and unauthenticated routes, while reusable UI components will be used for listing cards, filter badges, and request status views. Expo modules will be used for location permission and map redirection, and media upload will support property image handling. The deployment pipeline will use Expo development builds and production Android package generation.

3.2.2 Backend / Database

The backend will use Supabase as a cloud-hosted system from the initial deployment. Supabase PostgreSQL will store users, properties, booking requests, and reviews using foreign keys and indexed fields for filtering and scalable query performance. Supabase Auth will manage secure registration and login with role-based access for tenants and landlords, and Row Level Security (RLS) policies will enforce record-level authorization. Supabase Realtime will provide live booking-status updates, Supabase Storage will handle property images, and Edge Functions will implement server-side validation, business rules, and future notification workflows.

3.3 Module Implementation

3.3.1 Authentication and Onboarding Module

Authentication starts when the user launches the application and enters his phone number, with Nepal being (+977) the default country code. Clerk, the third party identity provider is used here which sends a 6-digit One-Time Password (OTP) via SMS. The user enters the OTP, and Clerk authenticates it. Clerk was chosen because it provides out-of-the-box sign-in UI and takes care of session management, thus there is no need to implement authentication from scratch. After validating the phone number, Clerk generates a session token (JWT), which gets sent to Supabase. Supabase validates the token in every incoming request and utilizes Row-Level Security (RLS) policy in order to allow users to access only his data. If the user is logging in for the first time, the system validates the `onboarding_complete` field in his profile and if it is false, prompts the user through a quick onboarding flow. After that, they choose their role (Tenant or Landlord) by clicking on the appropriate card instead of selecting from the dropdown

list. Then user enter their name and select the city, and in case of tenants, they choose property preferences. After completing this onboarding process, the value of `onboarding_complete` is set to true, and the user is redirected to their respective homepage. The data captured during this process is saved in three different tables: `profiles`, `user_roles`, and `user_preferences`.

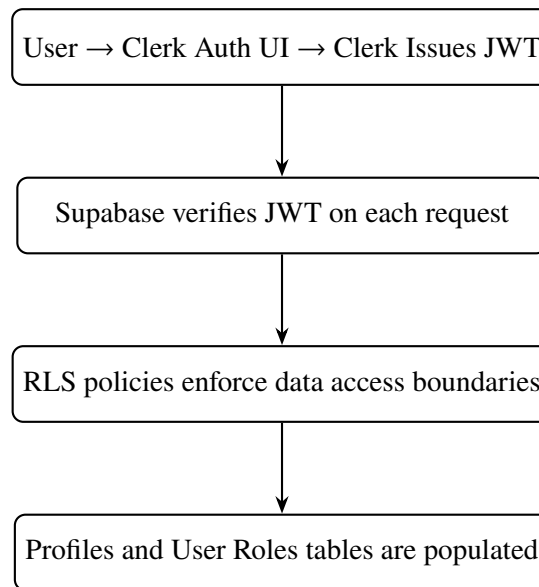


Figure 3.3: Authentication and Onboarding Workflow

3.3.2 Property Listing Management Module

Listings are generated by landlords using a structured multi-stage form rather than a single-page form. First, the landlord picks the type of the property as Apartment, House, Room, or Studio, and this determines what fields will be available in the next step, because certain fields like number of bedrooms or floors apply only to Apartments, whereas certain tenant preference fields like gender restriction, student-friendly, or couple allowed apply to Rooms. Then, the landlord provides the pricing information, including the price per month, the negotiability of that price, and the earliest possible move-in date, after which he/she provides the location information in terms of address, city, locality, and optionally the latitude/longitude of the property so that it will appear on the map view in the future. After that, he/she selects certain amenities out of a predefined list like WiFi, parking, and water supply, writes a brief description, and uploads multiple images into Supabase Storage with one selected as the cover photo.

In total, there are six steps and throughout them, form submission is kept in a single state manager built around React Context so that nothing is erased if the user decides to change their mind about a certain step and goes back to it later.

3.3.3 Property Discovery Module

Tenants have three different options to look for listings: The first one is the home feed which is the most popular listings displayed based on a combination of recent activity, tenant preferences, and recommendations from the AI engine. Also, the tenants are being provided with a search and filter option where the tenants can filter search results for keyword, property type, price range, property size by minimum bedrooms, location, property-status(furnished/unfurnished) and move-in date. each filter acts as a query clause against the listings data. Filtering conditions are matched as query conditions to the listings table. In the map view, listings that have coordinate information are shown as pins on a map. Tapping a pin will open a card with a full listing link. In any of these views, when a listing is opened, it redirects to the detail page showing the complete image gallery and property details, landlord rating, and visit-request button.

3.3.4 Visit Request Workflow Module

Visit request procedure begins from the listing detail page, where the tenant chooses the date and time and makes the request, creating the request in the status Pending. The landlord, in turn, receives the request on his dashboard and accepts it, rejects with the option of specifying the reason, or offers a new date and time as part of the reschedule. Thus, returning the request to the status Pending to give the response to the tenant. Having accepted the request, the landlord has the opportunity to send two more types of information. They are the precise pin location of the property and a contact phone number, but those stay hidden until acceptance as a part of protecting the privacy of the landlord by default. When the visit has been completed, the request can be changed into the status Completed and allows the tenant to write a review, whereas the tenant can cancel the request at any moment prior to its completion.

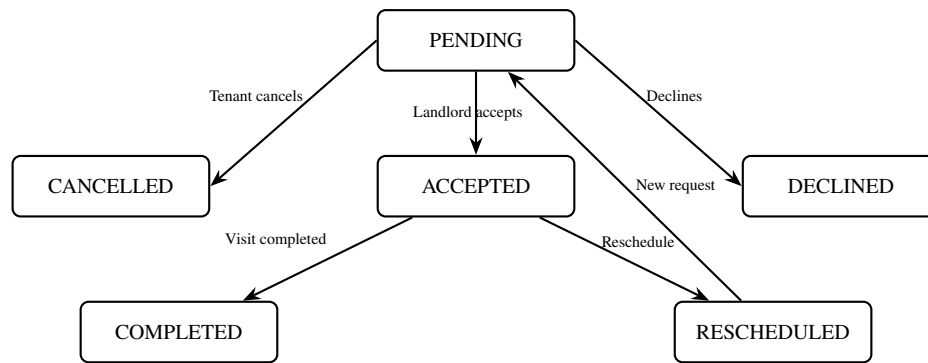


Figure 3.4: Visit Request Status Lifecycle

3.3.5 Landlord Experience Module

The landlord module of the app is a four-tab bottom navigation bar with options such as Dashboard, Requests, Alerts, and Profile. The Dashboard is the primary landing screen for the landlord and has card-based listings for active listings, pending requests, and KYC verification status. In addition, there is a listing of the properties belonging to the landlord with an option to edit, deactivate, or declare that the property is being rented, along with previews of the visit request received for that particular property. By clicking on any particular property from this listing, the user will be able to access all the relevant information associated with that listing, including photo galleries and visit request history. This tab is used to show all the visit requests according to their status such as Pending, Upcoming, or Past.

3.3.6 Tenant Experience Module

Upon signing in, the user will enter their personal home feed that is created based on the preferences of the user and the recommendation signals, backed up by the fast filters and a search field. The separate Requests tab will display all the requests of the user sorted chronologically along with the status of the request, and once the landlord approves the request and sends location and contact details to the user, the latter is informed about it and able to view the information in the request field. There is also the Saved Listings tab for the tenants that allows saving the properties of interest.

3.3.7 Pro Plan and Payment Module

This module processes the subscription payments that the landlords require to make their listings active. Since, eSewa is the most popular digital wallet in Nepal, we use it

as the payment gateway here. The subscription plans along with their cost and period of validity are stored in the form of a table named products so that they could be modified in the future without changing the code. Once a landlord selects a plan, the application creates a new transaction whose status will be Pending for now and sends him/her to the eSewa payment portal. After making payment through eSewa, the user is redirected back to BasoBas where the Supabase Edge Function verifies this payment by making an API call to the eSewa server. If verification succeeds, then the transaction's status changes to Complete and a new user_passes entry is made with the start and expiry dates, Otherwise, the transaction's status changes to Failed. To prevent being charged twice due to problems in the network, every transaction is processed with the use of the idempotency key, while the webhook listener serves as the official authority on the matter as to whether the transaction was successful or not. Prior to activating the listing, the system confirms that the landlord has a valid pass and not expired one.

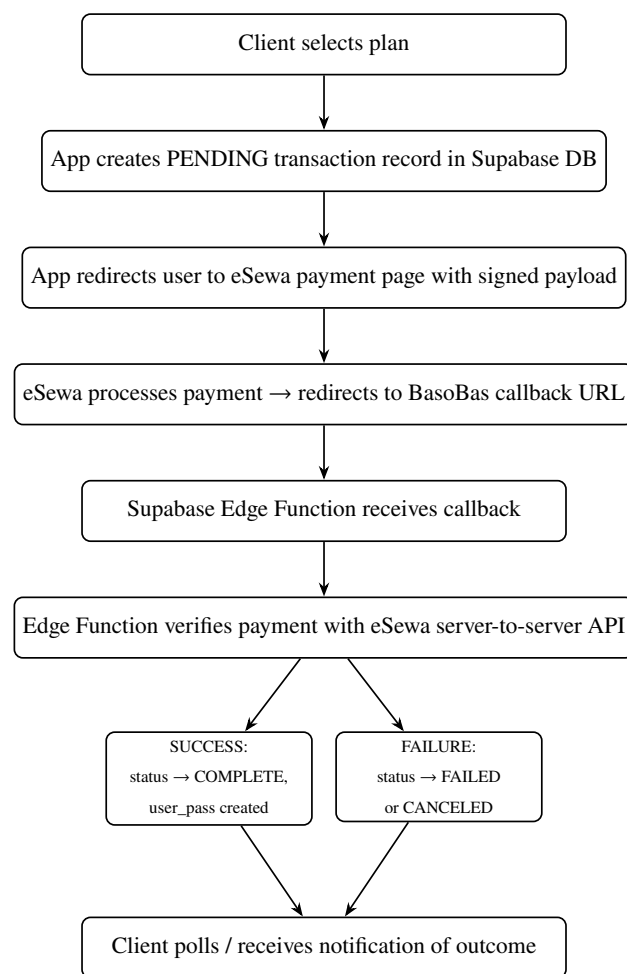


Figure 3.5: eSewa Payment Transaction Flow

3.3.8 Notification System Module

A set of trigger events was also outlined beforehand, which include such triggers as a new request to visit, request acceptance/rejection/rescheduling, sharing the location/contact, KYC being changed, payment being completed, or expiration of a pass. Each time any of these triggers takes place, a notification object is added to the notifications table, and Supabase Realtime is utilized to make sure that the notifications are available instantly on the user's device without the need for constant checks from the application itself. In case of notifications that have to be delivered to the user even when the application is closed, the use of Supabase Edge Functions will result in delivering push notifications via the push notification service on the device. The navigation menu of tenants/landlords contains an Alerts tab where the history of notifications is shown along with unread/read status.

3.3.9 Ratings and Reviews Module

The tenant can provide a review only if his or her visit request status is completed, in order to prevent any faked reviews by those tenants who have not even seen the property. Upon the submission of the review, the landlord's average rating is updated according to the running-average algorithm, and thus there is no need for recalculating the rating from scratch every time, as well as the whole process is made through one database transaction in order not to allow two concurrent reviews to corrupt the average calculation. The final rating along with the review count are displayed as a star rating. The aggregate rating is computed as a running average using the formula:

$$\text{new_avg} = \frac{(\text{old_avg} \times \text{total_reviews}) + \text{new_rating}}{\text{total_reviews} + 1}$$

3.3.10 KYC Verification Module

The KYC process involves automation via AI and human-in-the-loop in case of ambiguities. The process starts with selecting the document type (Citizenship Certificate or National ID), uploading images of both sides of the document and optionally an electricity bill, which acts as additional proof of address and generates a submission with status 'Under Review'. The documents are uploaded to a private and controlled access bucket in Supabase Storage service and the paths to the file are added to the database. Next, a Supabase Edge Function runs the AI verification pipeline which in-

volves reading the text from the document using OCR, verifying the document for any signs of forgery, comparing the document image to the selfie, checking consistency of the extracted information and returns a confidence score from 0 to 1. When the score is high enough, the submission gets status 'Verified' or 'Rejected', while lower score results in flagging and sending the submission to the Admin Portal. The administrator then reviews the documents, AI conclusions and the confidence score before making the final decision. If the submission is rejected, the administrator can also provide the reason for the rejection. This decision will make changes to both the submission history and the landlord's profile for consistency throughout the app. The landlord is informed about their decision, and if it is rejected, they have an opportunity to resubmit corrected information. This process continues until all attempts are made by the landlord.

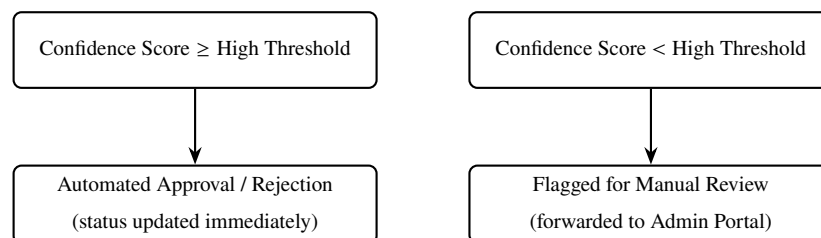


Figure 3.6: Confidence-Based Review Routing

3.3.11 AI Recommendation Module

Recommendation engine functions based on explicit feedback from tenants such as their preferences for certain properties, saved searches and visit requests. In addition to that, it analyzes listing signals such as property type, price, amenities, location, landlord's score which are considered as quality signals. All the above signals are analyzed using a hybrid ranking system where the listings that are similar to what tenant likes before get ranked higher using content based filtering, and the listings that are popular among other tenants with similar preferences get ranked higher using collaborative filtering.

3.3.12 Profile and Account Management Module

On the Profile Screen, there is a profile photo of the user, name, phone number, and city, but the Clerk ID underneath is not changeable. Photos uploaded on the avatar are saved in Supabase Storage and follow a uniform naming convention such that the older photo will be replaced easily by a new one uploaded. A user who is registered to both tenant

and landlord roles can toggle between roles from the profile screen, and the application will render the appropriate navigation and home screen without any need for logout. For those who have logged in as the landlord role, the profile screen also has a KYC Status panel, Active-pass panel containing information about the plan currently being used, and the expiry date, and Rating information. The Settings screen completes this module with a toggle button to automatically share the phone number after accepting the visit request, notifications options, and account deletion process.

3.4 Testing Strategy

Testing in BasoBas will be executed continuously across each iteration rather than only at the end of development. Every feature increment (authentication, listing, search, booking, review, and maps) will include test case preparation, execution, bug logging, and re-testing. This approach is intended to reduce late-stage integration issues and maintain release readiness throughout the 14-week timeline.

For mobile validation, the React Native (Expo) client will be tested on Android emulators and at least one physical Android device to verify UI behavior, navigation flow, form validation, and permission handling. For cloud services, Supabase-based modules will be tested for data integrity, API response correctness, Row Level Security enforcement, and realtime status updates for booking requests. Test cases will be traced to functional requirements so that each major workflow has clear pass/fail criteria.

The testing scope will include:

- **Unit Testing:** Validation utilities, distance calculation logic, filtering conditions, and rating-average computation.
- **API and Database Testing:** CRUD verification for users, properties, booking requests, and reviews with foreign-key and constraint checks.
- **Integration Testing:** End-to-end scenarios such as landlord posting a room, tenant discovering it through filters/nearby search, submitting a booking request, and receiving status updates.
- **Security and Authorization Testing:** Role-based access checks, unauthorized update attempts, and policy verification for protected records.
- **Usability and Robustness Testing:** Behavior under denied location permission, weak network conditions, invalid inputs, and repeated search requests.
- **User Acceptance Testing:** Guided trials with representative students and landlords to collect practical feedback on clarity, speed, and usefulness.

4. TASKS ACCOMPLISHED

BasoBas is a mobile-first rental accommodation finder built for students, working professionals, and small families searching for rental rooms and flats in cities. The following tasks have been accomplished till now as part of the BasoBas MVP development.

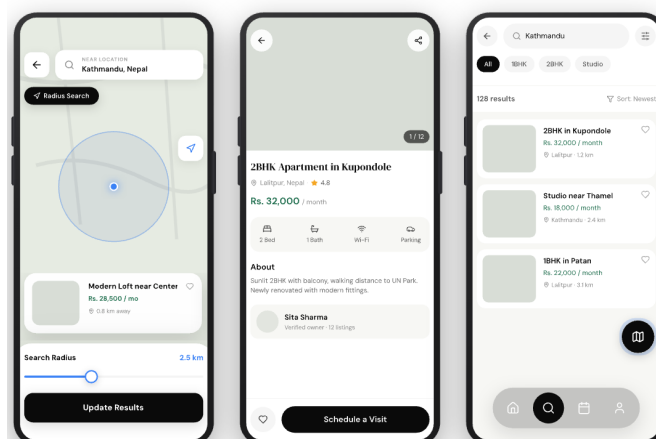
4.1 Product Planning and Documentation:

A considerable amount of time was spent on planning and documentation of the product before implementing it. Creating a solid base in the beginning helped avoid any misunderstandings, miscommunication, and confusion during development phase. The product requirements document that includes all the features, user flows, screens, data model, and business rules of the product was written. This document serves as the reference point for all development decisions throughout the development lifecycle. In addition, the design system was completely described. It includes the colors, typography, spacing system, component library, and interaction pattern. In this way, it becomes very simple to take any design decision that fits the product's whole vision. As a result, all the screens in the application will have the same unified. The user journey was also thoroughly mapped out. It includes from the very first time a user opens the app up to the closing of a rental, covering edge cases, error states, and alternative paths that a particular user might follow. All application screens were prepared and organized systematically into ten development phases. At last, forty-eight screens were identified, documented, and categorized according to their functional purpose within the system.

4.2 UI/UX Design:

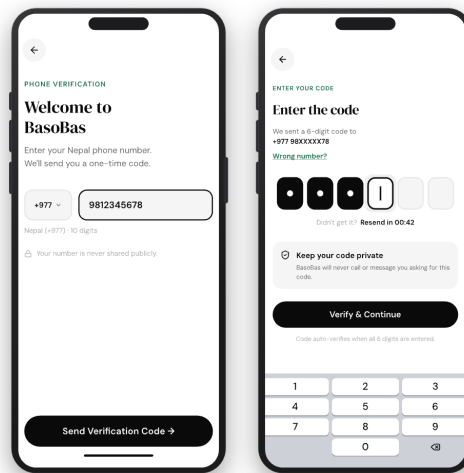
The whole design of the application has been done in Figma. There are 48 screens in total which have all been designed. The design of the application is minimalistic in nature. The design employs the white-based color theme to reflect the trustworthy nature, forest green as the brand accent color symbolizing stability and growth, premium fonts with DM Serif Display being used for headings and DM Sans for the UI text. The design features a very bright, trustworthy color of white as a primary color. The sole brand color apart from that is forest green, a shade which means stability and growth.

For the premium feel of the design, headlines and interface texts are done in different fonts which are DM Serif Display and DM Sans respectively. Also, there are lots of empty spaces which make the app look fresh and clean. We have created seven screens that guide the users from authenticating themselves to the whole onboarding of a user. These include screens like landing screen, phone number submission, OTP verification, user account creation, profile details setup, and the final phase of identity verification. We have also designed the full tenant experience. It includes the home screen with an AI-recommendation section, search and filter interface, a large-screen map for property pins, property details view, visit scheduling view, visit tracking, a notifications feature, saved properties section, profile management section. We have also fully designed the whole landlord experience, covering landlord dashboard with quick stats, property management screens, a four-step listing creation flow, visit request management, and comparison screens. A full upgrade from a Basic plan to a Pro plan flow has been created. It includes the features comparison screen with the payment option, a payment receipt screen as well as a payment failure screen, and locked feature indicators throughout the app. A major change in UI of BasoBas app is floating dock navigation. Instead of a standard full-width bottom nav bar, the app uses a floating black frosted glass round pill that floats above the screen content. The app looks luxurious and contemporary with floating dock and at the same time the interface remains clean and not visually crowded.



4.3 User Authentication:

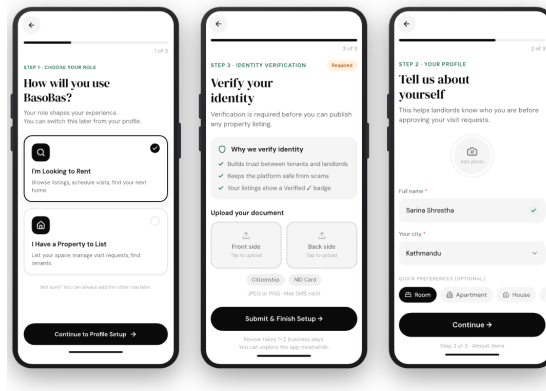
BasoBas implements phone-number-based One-Time Password (OTP) authentication using Clerk, a third-party identity management platform, integrated with Supabase via its native third-party authentication provider mechanism. This approach was adopted to simplify onboarding by eliminating the need for password creation, email registration, or social account linkage, while allowing Clerk-issued session tokens to be verified directly by the backend without a shared secret. The authentication interface includes a searchable country code selector with Nepal (+977) as the default. Upon submitting a phone number, users receive a six-digit SMS verification code valid for sixty seconds, with automatic submission upon completion. The system distinguishes between existing and new users to route them to the appropriate flow. Session tokens are persisted in the device's secure storage, all communication occurs over encrypted HTTPS, and rate-limiting is enforced to mitigate brute-force attempts.



4.4 Onboarding Flow Implementation:

After authentication, a new user will have to go through the well-structured onboarding process that collects minimum information about user to create a profile. Firstly, the system asks the user to select a role between Tenant and Landlord. Instead of using a simple dropdown menu, the interface presents the options as large, tappable cards with clear icons and brief descriptions. This makes the selection process more engaging. Secondly, users give the detail of their full name and pick the nearest city from a list

of Nepal cities and urban districts. Also, the user have to upload his or her photo and select property preferences. It is essential to provide a full name and city as landlords will need to know who is visiting their property. Thirdly, the system asks the user to upload their identity document for verification. This is mandatory for landlords since they have to upload a photograph of the front and back sides of their Nepal Citizenship Certificate or National Identity Card before they can post any listing. On the other hand, this process is optional for tenants. Verified tenants get a trust badge for landlords, which enables quick approval of their visits, but it is not mandatory to use the application. It is simple because, without verifying landlords, tenants cannot know that the people they are visiting are who they claim to be. Users can upload a picture either by taking a picture through the camera or choosing one from the photo library in the device. Pictures uploaded in this way are always compressed during the upload process to save data usage. By the mid-defense, the document uploading feature has already been developed. The backend verification, where pictures and documents will be evaluated and approved, will be ready in the next phase of development.



4.5 Dual-Role Account System:

On BasoBas, all users have a double-role account system. This means a single registered account simultaneously holds both a Tenant role and a Landlord role. In many real-life situations, there are several cases in which the same person may be renting a room themselves while also sharing a space they own. The double role account system is designed in such a way as to accommodate the situation easily. Users can switch between

the two roles instantly from the Profile screen. They do not need to log out, create a new account, or go through a separate registration process. This makes the platform flexible and reduces unnecessary complexity for users who serve both functions.

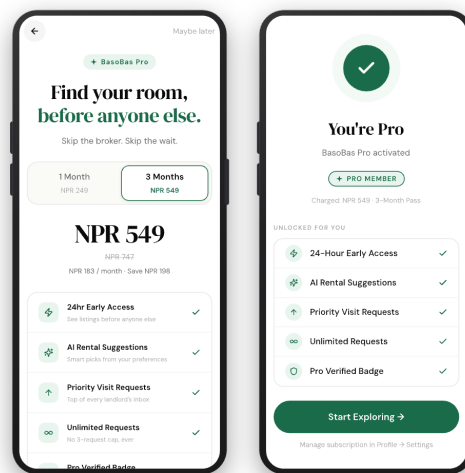
4.6 Database Architecture with Supabase:

The entire relational database design and implementation is done with Supabase and uses PostgreSQL for the backend. The main tables include Profiles where all user data including the name, phone number, city, profile picture URL, and whether the person has gone through onboarding are stored. The User Roles table stores the roles assigned to each user such as Tenant, Landlord or Both. The Landlord Profiles table stores landlord details such as whether their account has been verified, average rating and the number of reviews received. The KYC Submissions table stores uploaded documents along with their approval status. Finally, User Preferences table stores the tenant's property preferences. Another critical decision in designing the BasoBas database is the use of two-level privacy protection on the property location. The property location information is split into two different levels. The first level is the general location which is visible to all users such as "Talchhikhel, Lalitpur". The second level is the exact street address and the GPS coordinates separately stored which is only revealed to tenants who have been approved for a visit by the landlord. This two-level privacy protection is designed to protect the privacy of landlords while still providing enough information for tenants to find properties in their desired area. This privacy feature is achieved in the database using row level security policy. There are two storage buckets configured as well. First one is Avatars bucket for storing profile pictures and second one is KYC Documents bucket for identity verification documents.

4.7 eSewa Payment Gateway Integration:

eSewa is the most popular digital payment gateway in Nepal with seven million active users and hence the most suitable payment gateway for Nepal. So, it has been added to make payment for the subscription of the Pro plan. The Pro plan is designed as a complete replacement of the broker who charges too much amount of money from the tenant while offering only NPR 249 per month for the Pro plan with several advantages. The five features of the Pro plan include early listing of the property for 24 hours, full AI-based suggestions based on preference setup, top priority for visit requests at the

top of the landlord's request queue, unlimited visit requests instead of the three offered in the free version, and the gold Pro Verified badge for the user's profile and all the visit requests. The flow of payments is carried out by directing the user to the eSewa payment interface, making the payment using the eSewa account. Then validating the authenticity of the payment via the eSewa verification API in the backend to enable users to access Pro features. Also, the system has implemented various measures in case of network failure such as idempotency keys to ensure that the user is not charged twice, the use of the payment state machine, which is in the states initiated, pending, success, failed, and expired states. The backend also includes a webhook listener that acts as the primary source for confirming payment completion.



5. TASKS REMAINING

The following are the tasks which need to be accomplished.

- Property Listing Creation Flow
- Property Discovery and Search
- Visit Request Workflow
- KYC Verification Backend
- Landlord Request Management
- Push Notification System
- Ratings and Reviews
- AI Recommendation Engine
- Profile and Settings Screens
- Map Integration
- Testing and Quality Assurance
- Deployment and Release

REFERENCES

- [1] Central Bureau of Statistics Nepal. National population and housing census 2021: National report, 2023. URL <https://censusnepal.cbs.gov.np/>. Accessed: 2026-03-03.
- [2] S. Indrayani and S. Istikomah. Integrated web-based platform for boarding house rental services: Information architecture and booking workflow design. *Bit-Tech (Binary Digital Technology)*, 8(3):3834–3845, April 2026. doi: 10.32877/bt.v8i3.3722.
- [3] Nicole Ozer. Location-based services: Time for a privacy check-in. ACLU of Northern California Issue Paper, 2010. URL <https://www.aclu.org/news/privacy-technology/location-based-services-time-privacy-check-in>. Accessed: 2026-03-03.
- [4] S. S. Diaz, J. R. M. C. Layos, C. A. O. Pagaduan, R. L. G. Sarmiento, A. G. T. Sison, and E. A. Serrano. Roommeet: A renting mobile application. In *Proceedings of the 2024 9th International Conference on Intelligent Information Technology (ICIIT)*, pages 564–571, Ho Chi Minh City, Vietnam, February 2024. doi: 10.1145/3654522.3654606.
- [5] Audun Jøsang, Roslan Ismail, and Colin Boyd. A survey of trust and reputation systems for online service provision. *Decision Support Systems*, 43(2):618–644, 2007. doi: 10.1016/j.dss.2005.05.019.
- [6] A. Farooq, I. Hussain, H. Iman, N. Kiran, N. I. Kajla, and M. A. Jamil. Roommate: An ai optimized space rental system. *Journal of Statistical Computing and Interdisciplinary Research*, 7(1):51–65, 2025.
- [7] C. Hess, A. Acolin, R. Walter, I. Kennedy, S. Chasins, and K. Crowder. Searching for housing in the digital age: Neighborhood representation on internet rental housing platforms across space, platform, and metropolitan segregation. *Environment and Planning A: Economy and Space*, pages 1–21, 2021. doi: 10.1177/0308518X211034177.

- [8] I. Zaman, M. M. Khan, T. Rahman, and M. S. Rahman. An interactive web platform and android application based on room sharing service for students. In *Proceedings of the 2021 IEEE World AI IoT Congress (AIIoT)*, pages 0127–0131, 2021. doi: 10.1109/AIIOT52608.2021.9454175.
- [9] G. Tyagi, T. Kumar, T. Tyagi, R. Kumar, R. Maheshwari, and R. Kaushik. Leveraging conversational ai in an intelligent room rental platform. *Computer Research and Development*, 25(5):247–256, 2025.
- [10] V. Balpande, A. Kumre, B. Dongarwar, H. Parate, T. Selukar, and P. Ganvir. Implementation & development of an application for room searching. *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, 12(4), April 2024. doi: 10.22214/ijraset.2024.60344.
- [11] A. Wakodikar, O. Thakre, and R. Yadav. Design and implementation of a to-let: A web based room finder application. *International Journal of Advanced Research in Science, Communication and Technology (IJARSCT)*, 6(6), March 2026. doi: 10.48175/IJARSCT-32064.
- [12] Fensen, E. Dewayani, and Wasino. Website application for locating and reserving house and boardroom for rent based on selected location or region. *IOP Conference Series: Materials Science and Engineering*, 1007:012112, 2020. doi: 10.1088/1757-899X/1007/1/012112.
- [13] O. J. Uchechukwu and A. A. Shehu. A potential online platform that aims to make renting and buying properties easy and hassle-free for foreigners in russia. *ScienceOpen Preprints*, May 2025. doi: 10.14293/PR2199.001679.v1.
- [14] Jean-Charles Rochet and Jean Tirole. Platform competition in two-sided markets. *Journal of the European Economic Association*, 1(4):990–1029, 2003. doi: 10.1162/154247603322493212.
- [15] E. F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970. doi: 10.1145/362384.362685.
- [16] Roger W. Sinnott. Virtues of the haversine. *Sky and Telescope*, 68(2):159, 1984.

- [17] Steven Tadelis. Reputation and feedback systems in online platform markets. *Annual Review of Economics*, 8(1):321–340, 2016. doi: 10.1146/annurev-economics-080315-015325.
- [18] Teresa Scassa and Anca Sattler. Location-based services and privacy. *Canadian Journal of Law and Technology*, 9(1), 2011. URL <https://digitalcommons.schulichlaw.dal.ca/cjlt/vol9/iss1/6/>.